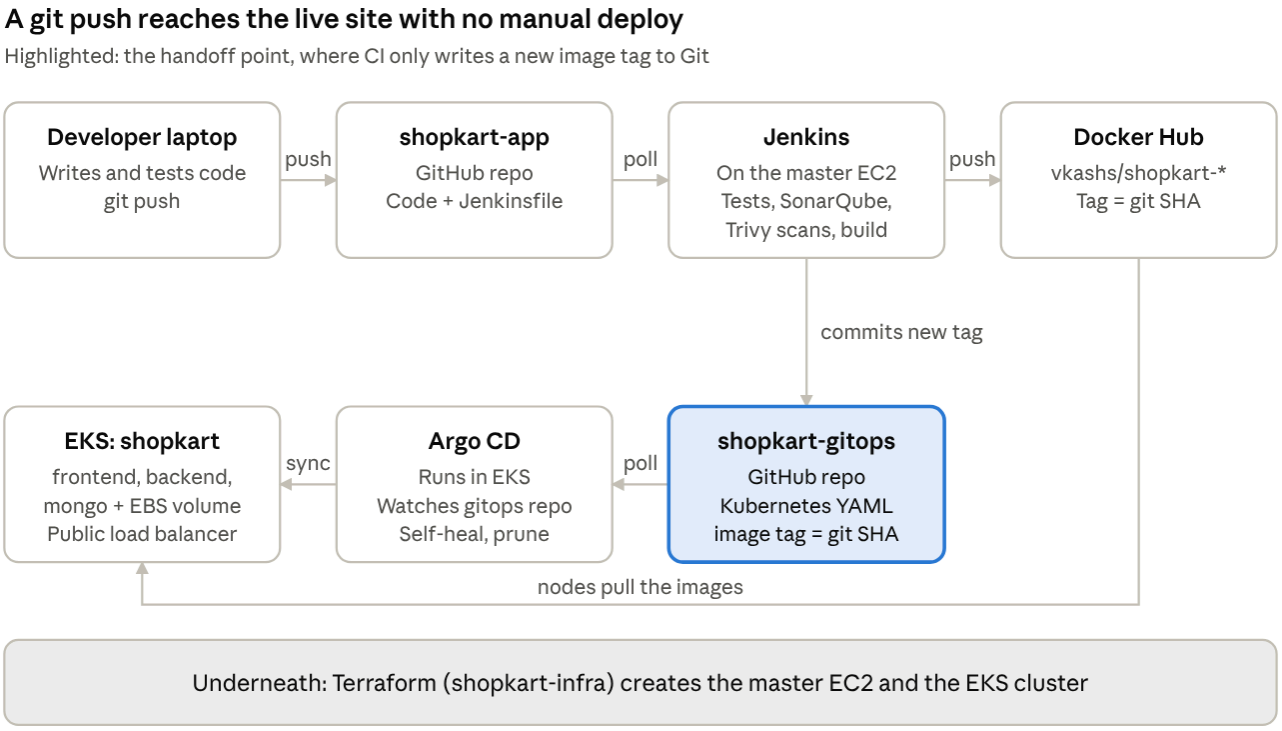
ShopKart DevOps Project: Build Guide

Oct 30, 2026 · @vikash

1. Overview

ShopKart is a small online shop app (React frontend, Node.js backend, MongoDB) that we took from code on a laptop to a live website on AWS EKS, with every step automated. A `git push` builds, tests and scans the app, publishes the images, and Argo CD deploys them to Kubernetes. Nobody runs a manual deploy.



ShopKart flow · laptop to live site, 7 parts on a Terraform base

Top row: the code goes to GitHub, Jenkins builds and scans it, and the images go to Docker Hub. Bottom row: Jenkins writes the new tag to the GitOps repo, Argo CD applies it, and the EKS nodes pull the images.

The stages we built, in order

Stage	What we built	Where it runs	Status
1	The app and its containers	Laptop (docker compose)	Done

Stage	What we built	Where it runs	Status
2	A "master" EC2 server, created with Terraform, with Jenkins and SonarQube	AWS EC2	Done
3	A Jenkins pipeline: test, analyse, scan, build, push	Master EC2	Done
4	An EKS Kubernetes cluster, created with Terraform	AWS EKS	Done
5	GitOps delivery with Argo CD	EKS	Done
6	Monitoring with Prometheus and Grafana	EKS	Done

Main tools and their job

Tool	Its job in this project
Git + GitHub	Stores the code, the infrastructure and the Kubernetes files in 3 repos
Docker	Packages the frontend and backend as images
Terraform	Creates all AWS resources from code: the master EC2, the network and the EKS cluster
Jenkins	Runs the CI pipeline on every push
SonarQube	Checks code quality and blocks the build if the quality gate fails
Trivy	Scans the source and the images for vulnerabilities and secrets
Docker Hub	Stores the built images (`vkashs/shopkart-backend` , `vkashs/shopkart-frontend`)
Amazon EKS	Runs the app on Kubernetes
Argo CD	Keeps the cluster matched to the GitOps repo
Prometheus + Grafana	Metrics, dashboards and alerts (Stage 6)

Region: everything runs in `ap-southeast-2` (Sydney), the region allowed for this AWS account.

2. The three repos and how they connect

We split the project into three GitHub repos by purpose: what we build (app), what it runs on (infra), and what runs in the cluster (gitops). Each README has the same project map linking to the other two.

Repo	Contains	Who changes it	Who reads it
shopkart-app	`backend/`, `frontend/`, Dockerfiles, `docker-compose.yml`, `Jenkinsfile`, `docs/architecture.md`	Developer (me)	Jenkins, every 2 minutes
shopkart-infra	`infra-master/` (Jenkins EC2), `bootstrap/` (state bucket), `infra-eks/` (VPC + EKS)	DevOps (me), with Terraform	Terraform
shopkart-gitops	`manifests/` (Kubernetes YAML), `apps/shopkart-app.yaml` (Argo CD Application)	Jenkins (image tag) and me (config)	Argo CD, every 3 minutes

Since Stage 6, `shopkart-gitops` also holds `manifests/backend-servicemonitor.yaml` and `manifests/alerts-prometheusrule.yaml` (applied by Argo CD), and `platform/monitoring/values.yaml` (used by Helm, ignored by Argo CD).

How the repos link to each other

1. I push app code to **shopkart-app**.
2. Jenkins builds it and pushes new images to Docker Hub.
3. Jenkins edits the two `image:` lines in **shopkart-gitops** and commits them.
4. Argo CD sees that commit and updates the cluster.
5. **shopkart-infra** is separate: it creates the machines and the cluster that the other two run on. It changes only when the infrastructure changes.

Why GitOps has its own repo: the Jenkins commit with the new tag would otherwise trigger another Jenkins build in the app repo, in an endless loop. A separate repo also means every commit there is one deployment, so going back is a `git revert`.

Never stored in any repo: `terraform.tfvars`, Terraform state and plan files, passwords and tokens, `node_modules` and build output.

3. Stage 1: the app and its containers

Goal: a working 3-tier app that runs on the laptop with one command, packaged as containers, with the code in GitHub. The code was generated by an AI coding tool (Antigravity) from detailed prompts, then reviewed and tested.

What we installed on the laptop and why

Tool	Why
Git	Version control, push to GitHub
Node.js 24 (LTS)	Run and test the app locally; same version as in the Docker images
Docker Desktop + Docker Compose	Build images and run all three tiers together
AWS CLI	Talk to AWS from the laptop
Terraform	Create AWS resources from code (Stage 2)
Antigravity	AI coding tool used to generate code from prompts

Steps we followed

1. **Backend** (`backend/`): Node.js 24 + Express REST API for products, MongoDB through Mongoose, Prometheus metrics with prom-client, JSON logs, 4 unit tests.
2. Made the backend **start listening first and connect to MongoDB in the background with retries**, so it stays up while the database starts.
3. Added three health endpoints: `/health` (process alive), `/ready` (database connected), `/metrics` (for Prometheus).
4. **Frontend** (`frontend/`): React + Vite page to list, add, edit and delete products. It calls `/api/...` with relative paths, so the same build works anywhere.
5. **Containers:** a multi-stage Dockerfile for each part. The backend runs as user `node` . The frontend is served by a non-root nginx on port 8080, which also forwards `/api/` to the backend service `shop-backend:8080` .
6. **docker-compose.yml** runs Mongo, backend and frontend together, with healthchecks and a named volume for the data. The site opens at `http://localhost:3000` .
7. Tested: all three containers healthy, add/edit/delete works, and the data survives `docker compose down` and `up` .
8. Scanned the repo for secrets with gitleaks, created the GitHub repo `shopkart-app` , added an SSH key to GitHub, and pushed.

How the three tiers connect

From	To	How
Browser	Frontend (nginx)	HTTP on port 3000 locally, port 80 on EKS
Frontend (nginx)	Backend	nginx forwards `/api/` to `shop-backend:8080`
Backend	MongoDB	`MONGO_URL` environment variable, port 27017

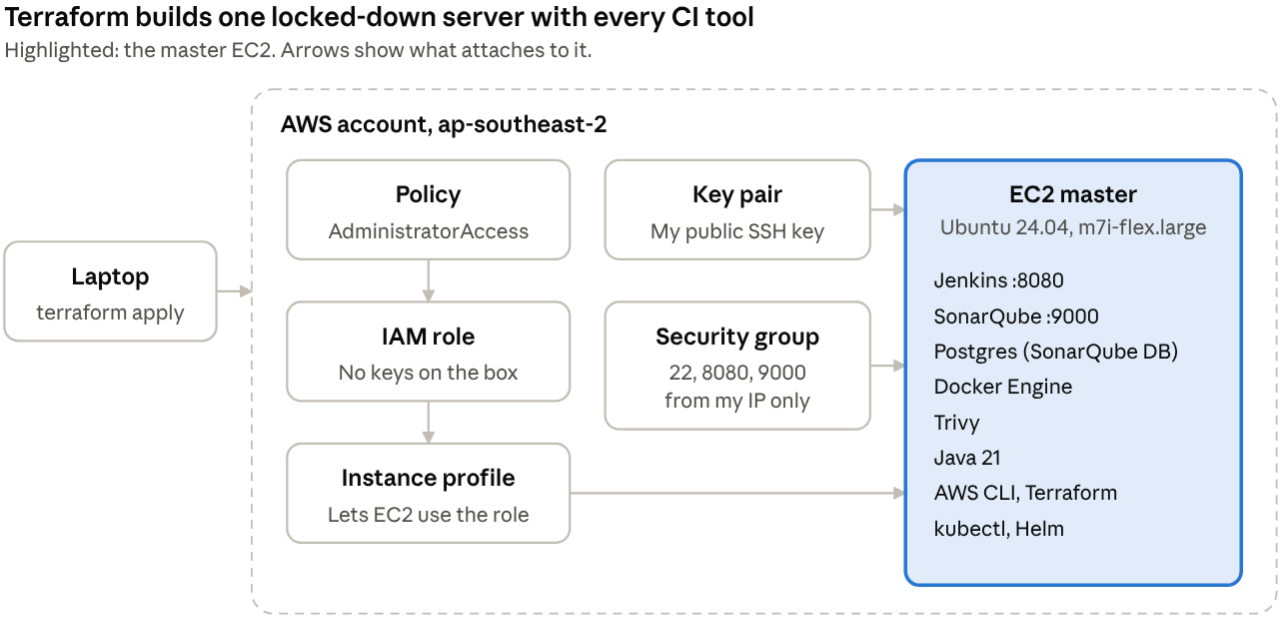
4. Stage 2: Terraform and the master EC2

Goal: one Linux server in AWS, created entirely by Terraform, that has every tool the pipeline needs. We call it the "master". Code: `shopkart-infra/infra-master/`, run from the laptop.

How we planned it

1. **Checked the AWS account first:** which identity we use, which region is allowed (`ap-southeast-2`), and which instance types the Free plan allows (`m7i-flex.large`, 2 vCPU, 8 GB).
2. Set a **budget alert** in AWS before creating anything.
3. Created a **separate SSH key** on the laptop for this server (`~/.ssh/shopkart-master`).
4. Wrote the Terraform, ran `terraform plan -out tfplan` to review it, then `terraform apply tfplan`.
5. Waited for the startup script to finish (`cloud-init status: done`), then checked every tool.

What Terraform creates and how the pieces link



Master EC2 · 6 Terraform resources and the tools installed

The policy gives the role its permissions, the instance profile attaches the role to the server, and the key pair and security group control who can log in.

Resource	What it is	Links to
IAM role	An AWS identity the server uses, so no access keys are stored on it	Gets permissions from the policy
Policy attachment	Gives the role `AdministratorAccess` (needed to create EKS from this server; a lab choice)	Attached to the role
Instance profile	The "holder" that lets an EC2 instance use the role	Attached to the EC2
Key pair	My **public** SSH key, uploaded to AWS	Attached to the EC2
Security group	Firewall: ports 22 (SSH), 8080 (Jenkins), 9000 (SonarQube), only from my IP	Attached to the EC2
EC2 instance	Ubuntu 24.04, `m7i-flex.large`, 30 GB encrypted disk, IMDSv2 required	Runs the startup script below

What the startup script (user data) installs, and why

The script runs once, on the first boot, and stops at the first error.

Installed	Why we need it
Java 21	Jenkins is a Java program
Jenkins (LTS)	The CI server, on port 8080
Docker Engine	Builds and pushes the images; also runs SonarQube. Users `jenkins` and `ubuntu` are added to the docker group
SonarQube 26.8 + Postgres 17 (in Docker)	Code quality server on port 9000, with its database. The DB password is generated at boot and never stored in Git
Trivy	Vulnerability and secret scanner, called by the pipeline
git, yq, jq, unzip	Checkout and file editing in the pipeline
AWS CLI v2	Talk to AWS using the server's IAM role
Terraform	Create the EKS cluster from this server (Stage 4)
kubectl, Helm	Manage the Kubernetes cluster (Stages 4 to 6)

How we connect to it

- **SSH:** `terraform output -raw ssh_command` prints the command with the current IP.
- **Jenkins:** `http://<master-ip>:8080`. **SonarQube:** `http://<master-ip>:9000`.
- If the startup script changes, we rebuild the server with `terraform apply -replace=aws_instance.master`, because the script only runs on the first boot.

5. Stage 3: Jenkins, its connections and the pipeline

Goal: every push to `shopkart-app` is tested, checked by SonarQube, scanned by Trivy, built into images and pushed to Docker Hub, and any failed check stops the build. The pipeline is the `Jenkinsfile` in the app repo.

Step 1: set up Jenkins

1. Open `http://<master-ip>:8080`, unlock it with the password from `/var/lib/jenkins/secrets/initialAdminPassword`.
2. Install the **suggested plugins** (Pipeline, Git, Credentials Binding and others).
3. Create the admin user.

Step 2: add the extra plugins

Manage Jenkins → Plugins → Available plugins:

Plugin	Why
NodeJS	Installs Node 24 for `npm ci` and `npm test`
SonarQube Scanner	Adds `withSonarQubeEnv` and `waitForQualityGate`
Pipeline: Stage View	Shows each stage as a box on the job page
Email Extension	Build emails (planned)

Trivy and Docker need **no plugin**: they are installed on the server, and the pipeline calls them with `sh`.

Step 3: add the tools

Manage Jenkins → Tools: NodeJS installation named `node24` (Node 24.x), and SonarQube Scanner installation named `sonar-scanner`. Both are downloaded automatically on the first build.

Step 4: connect Docker Hub

1. On Docker Hub: **Account settings → Personal access tokens →** a new token with **Read & Write** access and a 90-day expiry.
2. In Jenkins: **Manage Jenkins → Credentials → System → Global → Add**, kind *Username with password*, username `vkashs`, password = the token, ID **`dockerhub`**.
3. On Docker Hub: **Default privacy → Public** (the free plan allows only one private repo).

Step 5: connect SonarQube (two directions)

1. **Token:** in SonarQube, **My Account → Security → Generate token** (type *Global Analysis Token*). In Jenkins, add it as kind *Secret text* with ID **`sonar-token`**.
2. **Jenkins → SonarQube: Manage Jenkins → System → SonarQube servers**: name `sonar`, URL `http://localhost:9000`, token `sonar-token`. Localhost works because both run on the same server.
3. **SonarQube → Jenkins (webhook):** in SonarQube, **Administration → Configuration → Webhooks**: URL `http://172.18.0.1:8080/sonarqube-webhook/`. SonarQube runs in a container, so it reaches Jenkins through the Docker network's gateway IP, not `localhost`. The webhook tells Jenkins whether the quality gate passed.

4. Each app folder has a `sonar-project.properties` file with its project key (`shopkart-backend`, `shopkart-frontend`) and the coverage report path.

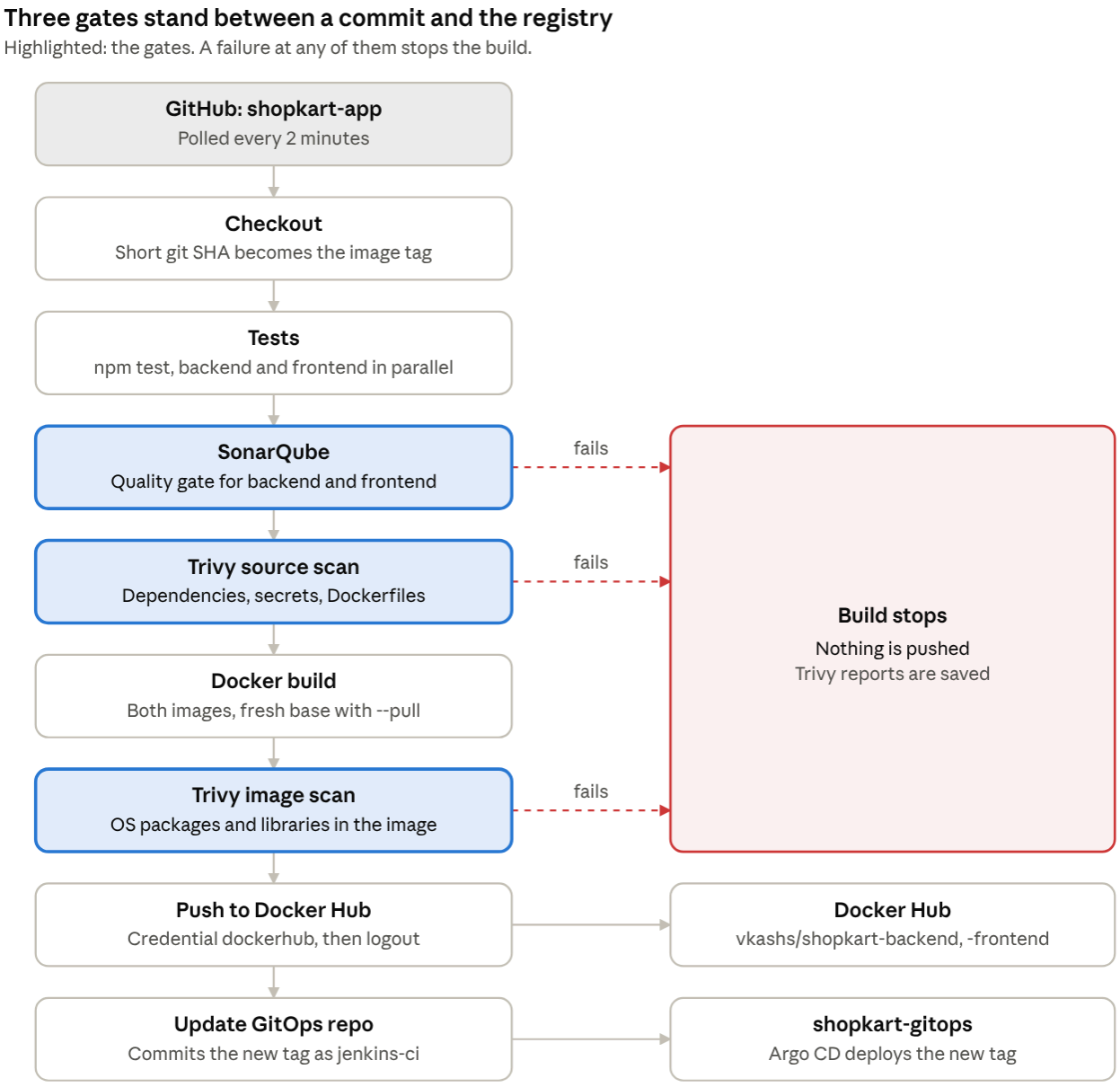
Step 6: connect GitHub

1. The app repo is public, so Jenkins reads it **without credentials**.
2. **New Item → `shopkart-app` → Pipeline → Pipeline script from SCM →** Git, `https://github.com/vkashsingh/shopkart-app.git`, branch `*/main`, script path `Jenkinsfile`.
3. The Jenkinsfile has `pollSCM('H/2 * * * *')`: Jenkins checks GitHub every 2 minutes. A GitHub webhook can't reach Jenkins because the firewall only allows my IP.
4. For Stage 5, a second credential **`github-token`** (a fine-grained GitHub token that can write only to `shopkart-gitops`) lets Jenkins push the new image tag.

Step 7: test the setup, then run the pipeline

A tiny `tools-check` pipeline printed the Node, Docker and Trivy versions and logged in to Docker Hub. Then we created the real job and clicked **Build Now** once to register the polling trigger.

The pipeline stages



Jenkins pipeline · 8 stages, 3 gates

Each stage runs only if the one above it passed. The three highlighted gates are where a quality or security problem stops the build before anything reaches Docker Hub.

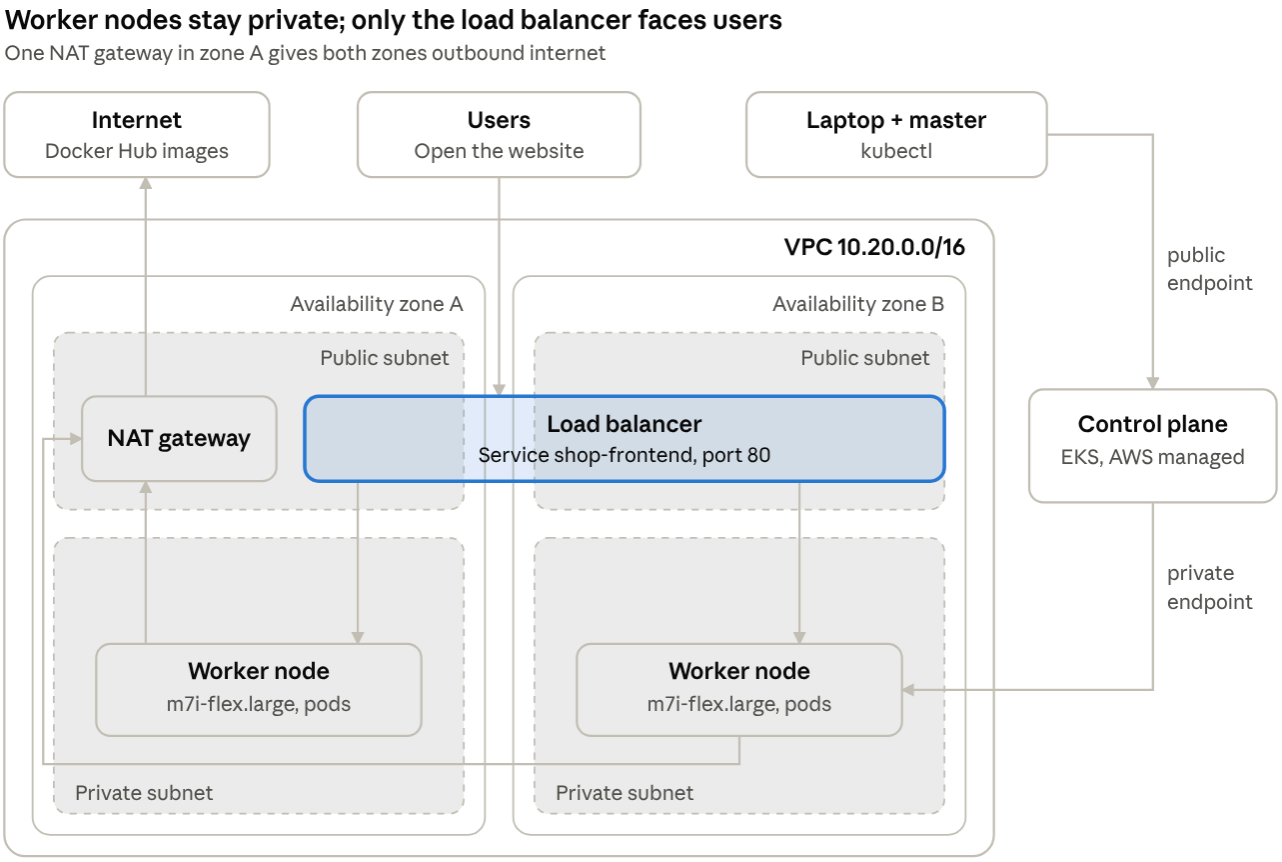
#	Stage	What it does
0	Checkout	Gets the commit. Its short git SHA becomes the image tag (for example `4c5123d`)
1	Tests	`npm ci` and `npm test` for backend and frontend, side by side

#	Stage	What it does
2	SonarQube backend	Sends the backend code and coverage to SonarQube, waits for the quality gate
3	SonarQube frontend	The same for the frontend
4	Trivy source scan	Scans dependencies, secrets and Dockerfiles; stops on CRITICAL
5	Docker build	Builds both images with `--pull`, so the base image is always fresh
6	Trivy image scan	Scans the finished images, including OS packages from the base image; stops on fixable CRITICAL
7	Push to Docker Hub	Logs in with `dockerhub`, pushes both images, logs out
8	Update GitOps repo	Writes the new tag into `shopkart-gitops` and commits it (added in Stage 5)
after	Cleanup	Saves the Trivy reports as build artifacts, removes the images, cleans the workspace

6. Stage 4: EKS with Terraform

Goal: a Kubernetes cluster (EKS 1.37) with 2 worker nodes in a private network, storage for MongoDB, and Terraform state kept safely in S3. Code: `shopkart-infra/bootstrap/` and `shopkart-infra/infra-eks/`.

How we planned it



EKS network · VPC, 2 zones, 4 subnets, 1 NAT

Users reach the pods only through the load balancer. The nodes have no public IPs: they pull images through the NAT gateway and talk to the control plane through its private endpoint, while only my laptop and the master can use the public endpoint.

Part	Decision
Kubernetes version	1.37, the newest on standard support (checked with `aws eks describe-cluster-versions`)
Network (VPC)	`10.20.0.0/16`, 2 availability zones, 2 public and 2 private subnets
Internet for the nodes	**One** NAT gateway (cheaper for a lab; production uses one per zone)
Worker nodes	Managed node group, 2 x `m7i-flex.large`, in the **private** subnets
Who can call the cluster API	Public endpoint limited to my laptop IP and the master IP; private endpoint on for the nodes

Part	Decision
Cluster access	Access entries: the master's role (creator) and my laptop role, both admin
Pod networking	VPC CNI add-on, installed **before** the nodes
Storage	EBS CSI driver add-on; StorageClass `gp3` as the default
AWS permissions for pods	**EKS Pod Identity** (the account blocks the IAM OIDC provider that the older IRSA method needs)
Terraform state	S3 bucket with versioning, encryption and native locking

Steps we followed

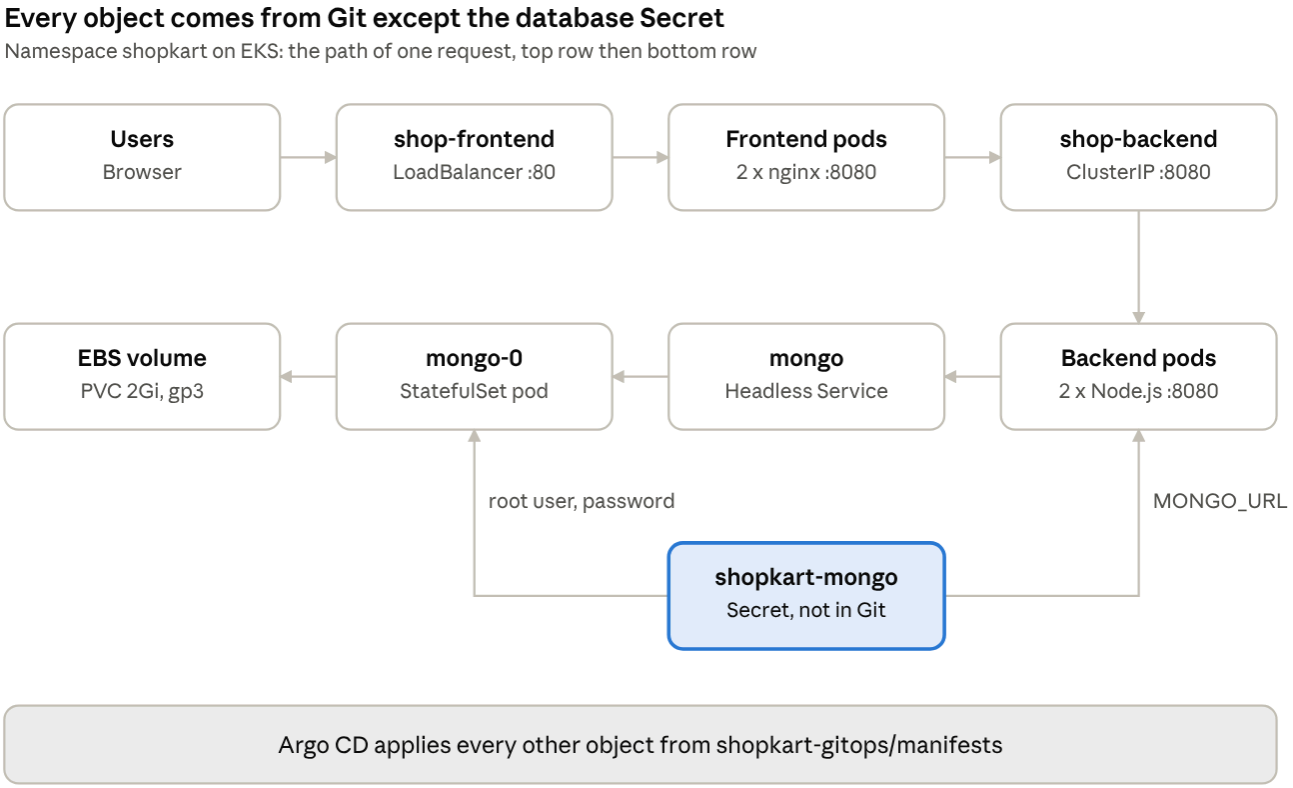
1. **Bootstrap (once):** `bootstrap/` creates the S3 bucket `shopkart-tfstate-<account-id>` for Terraform state. It uses local state itself, because the bucket must exist before any project can store state in it. Applied from the laptop.
2. **Write `infra-eks/`:** VPC module, EKS module, node group, add-ons and the Pod Identity role. The backend points to the S3 bucket, which is passed at `init` time so the account ID stays out of Git.
3. **Run it from the master server**, so Terraform uses the server's IAM role:
   a. `git clone` the infra repo on the master.
   b. Copy `terraform.tfvars` (my IP, the master IP, my laptop role) with `scp`; it is not in Git.
   c. Start `tmux`, so a dropped SSH connection doesn't stop Terraform.
   d. `terraform init -backend-config="bucket=..."`, `terraform plan -out tfplan`, `terraform apply tfplan` (about 20 minutes).
4. **Connect kubectl:** `aws eks update-kubeconfig --region ap-southeast-2 --name shopkart-eks`, on the master and on the laptop. On the laptop, the context was renamed to `shopkart-eks` so it is not confused with minikube.
5. **Apply the StorageClass:** `kubectl apply -f k8s/storageclass-gp3.yaml`.
6. **Check:** 2 nodes `Ready`; system pods running (`aws-node`, `kube-proxy`, `coredns`, `eks-pod-identity-agent`, `ebs-csi-*`); `terraform plan` says **No changes**; a test PVC becomes `Bound` and is then deleted.

What runs in `kube-system` and why

Add-on	Why
`vpc-cni` (`aws-node`)	Gives every pod an IP address. Without it nodes stay `NotReady`
`kube-proxy`	Routes Service traffic to pods
`coredns`	DNS inside the cluster (`mongo`, `shop-backend`)
`eks-pod-identity-agent`	Hands AWS credentials to pods that have a Pod Identity association
`aws-ebs-csi-driver`	Creates and attaches EBS volumes for PVCs, using its Pod Identity role

7. Stage 5: GitOps with Argo CD

Goal: the cluster always matches the `shopkart-gitops` repo, and a code push reaches the live site with no manual `kubectl`. Kubernetes files are plain YAML (no Helm), one object per file.



Namespace shopkart · 8 objects from Git, 1 Secret made by hand

A request goes from the load balancer to nginx, then to the backend, then through the headless `mongo` Service to `mongo-0`, whose data lives on an EBS volume. The Secret is the only object created by hand, so no password is ever stored in Git.

Step 1: write the Kubernetes files (`shopkart-gitops/manifests/`)

File	Creates	Notes
`00-namespace.yaml`	Namespace `shopkart`	Everything lives here
`mongo-statefulset.yaml`	MongoDB, 1 pod `mongo-0`	2Gi gp3 volume; user and password read from the Secret
`mongo-service.yaml`	Headless Service `mongo`	Gives the pod a stable DNS name
`backend-deployment.yaml`	2 backend pods	Image `vkashs/shopkart-backend:<tag>`; `MONGO_URL` from the Secret; probes `/ready` and `/health`
`backend-service.yaml`	Service `shop-backend` (ClusterIP)	Internal only, port 8080
`backend-pdb.yaml`	PodDisruptionBudget	At least 1 backend pod always up
`frontend-deployment.yaml`	2 frontend pods	Image `vkashs/shopkart-frontend:<tag>`
`frontend-service.yaml`	Service `shop-frontend` (LoadBalancer)	Creates an AWS load balancer: public port 80 to pod port 8080

Added in Stage 6: `backend-servicemonitor.yaml` (ServiceMonitor) and `alerts-prometheusrule.yaml` (PrometheusRule). See section 9.

All pods run as non-root users with limited permissions. `apps/shopkart-app.yaml` is the Argo CD Application that points at this folder.

Step 2: install Argo CD

1. `kubectl create namespace argocd`.
2. `kubectl apply -n argocd --server-side -f <Argo CD install.yaml>` (server-side, because Argo CD's definitions are too large for a normal apply).
3. Open the UI privately: `kubectl -n argocd port-forward svc/argocd-server 8443:443`, then `https://localhost:8443`.
4. Log in as `admin`. The first password is in the Secret `argocd-initial-admin-secret`.

Step 3: create the database Secret (outside Git)

`kubectl create secret generic shopkart-mongo` in namespace `shopkart`, with three keys: `mongo-root-username`, `mongo-root-password` (generated with `openssl rand -hex 16`) and `mongo-url`. The YAML files only refer to this Secret by name, so no password is ever in Git.

Step 4: connect Argo CD to the GitOps repo

`kubectl apply -n argocd -f apps/shopkart-app.yaml`. The Application tells Argo CD:

Setting	Value
Source	`https://github.com/vkashsingh/shopkart-gitops.git`, branch `main`, folder `manifests` (public repo, no credentials)
Destination	This cluster, namespace `shopkart`
Sync policy	Automated, with **prune** (delete what was removed from Git) and **self-heal** (undo manual changes)

After about 90 seconds the app showed **Synced** and **Healthy**, and the load balancer URL opened the site.

Step 5: connect Jenkins to the GitOps repo

1. On GitHub: a **fine-grained token** for `shopkart-gitops` only, permission *Contents: Read and write*, 90 days.
2. In Jenkins: credential kind *Username with password*, ID **`github-token`**.
3. A new last stage in the Jenkinsfile, **Update GitOps repo**: clone `shopkart-gitops`, replace the tag on the two `image:` lines with the new git SHA, commit as `jenkins-ci`, push.

Step 6: check the full loop

Check	What we did	What we saw
The site works	Opened the load balancer URL, added products	Data saved in MongoDB
Data survives a restart	Deleted pod `mongo-0`	It came back with the same volume and data

Check	What we did	What we saw
Git wins	Scaled the backend to 5 by hand	Argo CD put it back to 2
Full automation	Pushed a visible text change	Jenkins built `4c5123d`, committed the tag, Argo CD synced, the page changed

8. Connection map

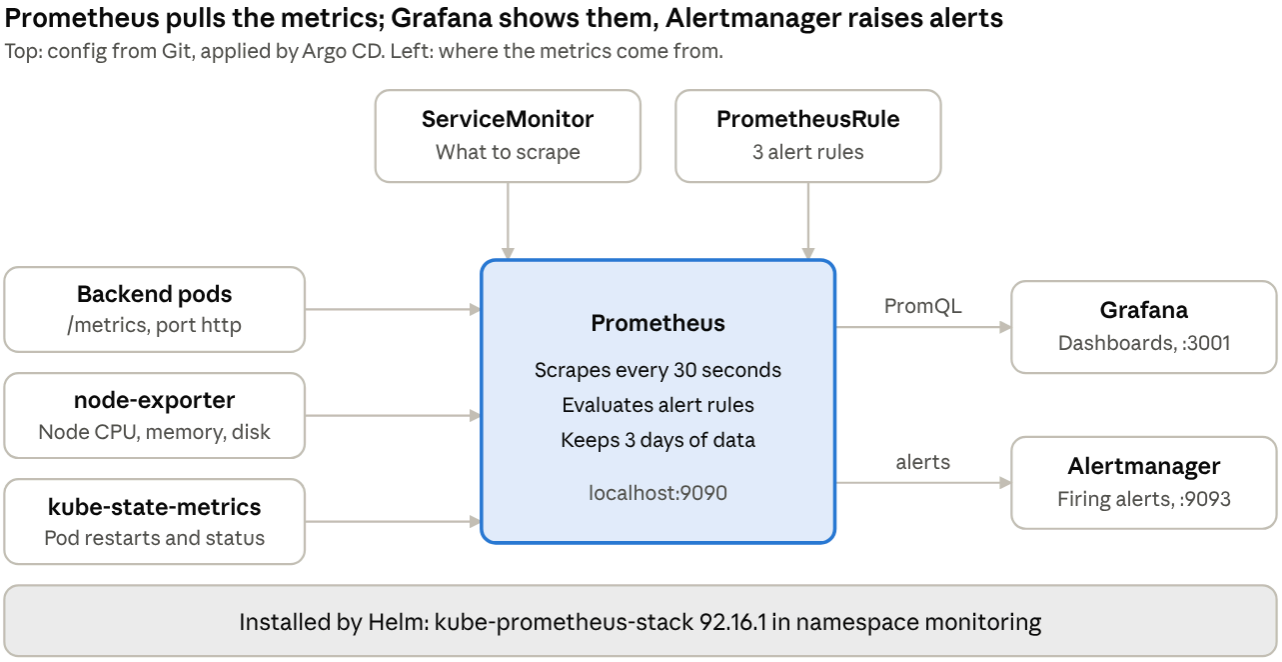
Every link between two tools in the project, in the order a change travels, with how it connects and what it authenticates with.

#	From	To	How it connects	Authenticates with
1	Laptop	GitHub (all 3 repos)	`git push` over SSH	SSH key `~/.ssh/id_ed25519` added to GitHub
2	Laptop	AWS	AWS CLI and Terraform	`aws login` (short-lived session)
3	Laptop	Master EC2	SSH; browser to :8080 and :9000	SSH key `shopkart-master`; security group allows my IP
4	Jenkins	GitHub `shopkart-app`	Polls every 2 minutes, `checkout scm`	None (public repo)
5	Jenkins	SonarQube	`withSonarQubeEnv('sonar')` to `http://localhost:9000`	Credential `sonar-token`
6	SonarQube	Jenkins	Webhook to `http://172.18.0.1:8080/sonarqube-webhook/`	Internal Docker network
7	Jenkins	Trivy	`sh 'trivy fs ...'` and `sh 'trivy image ...'`	None (local CLI)
8	Jenkins	Docker Hub	`docker login` then `docker push`	Credential `dockerhub` (read/write token)
9	Jenkins	GitHub `shopkart-gitops`	`git clone`, commit, `git push`	Credential `github-token` (fine-grained, one repo)
10	Master EC2	AWS (Terraform for EKS)	AWS API	EC2 instance role `shopkart-master-role`

#	From	To	How it connects	Authenticates with
11	Terraform	S3 state bucket	`backend "s3"` with `use_lockfile`	The instance role
12	Laptop and master	EKS API	`kubectl` after `aws eks update-kubeconfig`	EKS access entries; endpoint allows only the two IPs
13	Argo CD	GitHub `shopkart-gitops`	Polls every 3 minutes	None (public repo)
14	Argo CD	EKS	Applies the manifests in namespace `shopkart`	Runs inside the cluster
15	EKS nodes	Docker Hub	Pull `vkashs/shopkart-*:<tag>` through the NAT gateway	None (public images)
16	EBS CSI driver	AWS EBS	Creates and attaches volumes for PVCs	Pod Identity role `shopkart-ebs-csi-pod-identity`
17	Backend pods	MongoDB	`MONGO_URL` to `mongo:27017`	Secret `shopkart-mongo`
18	Users	Frontend	AWS load balancer, port 80	None (public site)
19	Laptop	Monitoring stack	helm upgrade --install with values.yaml; port-forward to Prometheus :9090 and Grafana :3001	kubeconfig (EKS access entry)
20	Prometheus	Backend pods	Scrapes /metrics every 30s, found through the ServiceMonitor	None (inside the cluster)
21	Grafana	Prometheus	PromQL queries through the built-in Prometheus data source	Grafana login: Secret grafana-admin
22	Prometheus	Alertmanager	Sends firing alerts from the PrometheusRule	None (inside the cluster)

9. Stage 6: Prometheus and Grafana

Goal: collect metrics from the cluster and from the backend's `/metrics`, show them on Grafana dashboards, and raise alerts when something goes wrong. Done: the backend is scraped (2/2 targets UP), 3 alerts are defined in Git, and one alert was fired on purpose. It runs on the existing EKS nodes, so it adds no AWS cost.



Monitoring · 3 metric sources, 2 config objects, 3 tools

The ServiceMonitor and PrometheusRule live in Git and tell Prometheus what to scrape and when to alert. Prometheus pulls metrics from the backend, the nodes and Kubernetes itself; Grafana queries it, and Alertmanager receives the firing alerts.

Decision: Helm for the monitoring stack only

The app stays plain YAML. Prometheus and Grafana are third-party software, so we installed them with the Helm chart **kube-prometheus-stack**, version **92.16.1**, the way you would use `apt install` on a server. The chart's settings live in Git at `shopkart-gitops/platform/monitoring/values.yaml`. That folder is outside `manifests/`, so Argo CD ignores it; Helm reads it.

What gets installed and why

Component	Why
Prometheus Operator	Manages Prometheus and turns `ServiceMonitor` and `PrometheusRule` objects into Prometheus config
Prometheus	Scrapes and stores the metrics (3 days), evaluates the alert rules
Grafana	Dashboards; about 25 come ready-made with the chart

Component	Why
Alertmanager	Receives firing alerts and routes notifications
node-exporter (one per node)	Node CPU, memory, disk and network
kube-state-metrics	Kubernetes object state: pods, restarts, deployments

Steps we followed

1. **Checked capacity:** `kubectl describe nodes` showed CPU and memory requests well under 60% per node, and 24 pods out of the 58 that 2 nodes allow.
2. **Installed Helm on the laptop:** `brew install helm`.
3. **Wrote the values file** `platform/monitoring/values.yaml` (key settings below).
4. **Created the namespace and the Grafana password Secret:** `kubectl create namespace monitoring`, then `kubectl -n monitoring create secret generic grafana-admin` with `admin-user=admin` and a password generated by `openssl rand -hex 16`.
5. **Installed the chart, pinned to a version:**
   a. `helm repo add prometheus-community https://prometheus-community.github.io/helm-charts`, then `helm repo update`.
   b. `helm upgrade --install monitoring prometheus-community/kube-prometheus-stack -n monitoring --version 92.16.1 -f values.yaml --wait --timeout 10m`.
6. **Checked the pods:** 7 pods. Grafana restarted with `OOMKilled` (exit code 137) at a 256Mi memory limit, so we raised it to 512Mi and ran the same `helm upgrade` again. `helm history monitoring` shows each revision.
7. **Opened the UIs privately with port-forward:** Prometheus `kubectl -n monitoring port-forward svc/monitoring-kube-prometheus-prometheus 9090:9090` (http://localhost:9090), Grafana `kubectl -n monitoring port-forward svc/monitoring-grafana 3001:80` (http://localhost:3001).
8. **Connected the backend:** added `manifests/backend-servicemonitor.yaml` to the GitOps repo. Argo CD applied it, and Prometheus showed the target `serviceMonitor/shopkart/shop-backend/0` as **2/2 UP**.
9. **Added the alerts:** `manifests/alerts-prometheusrule.yaml` with 3 alerts. Argo CD applied it, and the group `shopkart.rules` appeared in Prometheus.

10. **Tested an alert:** restarted one backend container 4 times with `kubectl -n shopkart exec <pod> -- kill 1`. `PodRestartingOften` went **Pending**, then **Firing** after 1 minute.
11. **Committed** the values file with the chart version in its header.

Key settings in `values.yaml`

Setting	Value	Why
`grafana.admin.existingSecret`	`grafana-admin`	The Grafana password is in a Secret, never in the file
Grafana memory	request 256Mi, limit 512Mi	256Mi was too little; Grafana was OOMKilled
`prometheus.prometheusSpec.retention`	`3d`	Enough for a lab, keeps disk and memory small
`serviceMonitorSelectorNilUsesHelmValues`	`false`	Prometheus picks up every ServiceMonitor, without needing a Helm release label
`ruleSelectorNilUsesHelmValues`	`false`	The same for PrometheusRules
`kubeEtcd`, `kubeControllerManager`, `kubeScheduler`, `kubeProxy`	disabled	EKS manages the control plane, so these can't be scraped and would show as DOWN

How the backend is connected (ServiceMonitor)

The ServiceMonitor `shop-backend` (namespace `shopkart`) selects the Service with label `app=shop-backend`, scrapes its port named `http` at path `/metrics`, every 30 seconds. Prometheus then scrapes each backend pod directly at its pod IP.

The three alerts (PrometheusRule `shopkart-alerts`)

Alert	Fires when	Severity
`BackendHighErrorRate`	More than 5% of backend requests return 5xx, for 5 minutes	critical
`PodRestartingOften`	A container in `shopkart` restarts more than 3 times in 10 minutes, for 1 minute	warning
`NodeMemoryHigh`	A node uses more than 85% of its memory, for 10 minutes	warning

An alert goes **Inactive → Pending** when its condition becomes true, and **Firing** only if it stays true for the whole `for:` time.

How to use it

- **Prometheus:** *Status → Target health* (what is scraped), *Alerts* (rule states), *Query* (PromQL).
- **Grafana:** log in as `admin`; the password is `kubectl -n monitoring get secret grafana-admin -o jsonpath='{.data.admin-password}' | base64 -d`. Useful dashboards: *Kubernetes / Compute Resources / Namespace (Pods)* with namespace `shopkart`, *Kubernetes / Compute Resources / Pod*, *Kubernetes / Compute Resources / Cluster*, *Node Exporter / Nodes*.
- **Alertmanager:** `kubectl -n monitoring port-forward svc/monitoring-kube-prometheus-alertmanager 9093:9093`, then http://localhost:9093.

Queries for the app's own panels

Panel	PromQL
Requests per second, by route	`sum by (route) (rate(http_requests_total{namespace="shopkart"}[5m]))`
Error rate	`sum(rate(http_requests_total{namespace="shopkart",status=~"5.."}[5m])) / sum(rate(http_requests_total{namespace="shopkart"}[5m]))`
95th percentile latency	`histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket{namespace="shopkart"}[5m])))`

Panel	PromQL
Restarts per pod	`kube_pod_container_status_restarts_total{namespace="shopkart"}`

What the dashboards showed

- `mongo-0` is the busiest pod even when idle (about 0.085 cores), mostly from its `mongosh` health probes.
- The backend pods use about 4% of their CPU request, so their requests could be lowered (right-sizing).

Optional next steps

- Build a *ShopKart Overview* dashboard from the queries above and save it in Git as a ConfigMap labelled `grafana_dashboard: "1"`, so Grafana loads it automatically.
- Stage 7: break-fix practice. Stage 8: teardown (section 10).

10. Running, pausing and tearing down

With everything up the project costs about $0.55 per hour; the master alone about $0.12. These are rough figures, so check current AWS pricing.

What costs money	About per hour
EKS control plane	$0.10
2 worker nodes	$0.24
NAT gateway	$0.06
Load balancer for the site	$0.03
Master EC2	$0.12

Pause the master overnight

1. `aws ec2 stop-instances --instance-ids <master-id>`. Jenkins and SonarQube settings stay on the disk.
2. Next time: `aws ec2 start-instances`, then `terraform apply -refresh-only` in `infra-master` to get the new public IP.
3. Put the new IP in **Manage Jenkins → System → Jenkins URL** and in `infra-eks/terraform.tfvars`.

Stopping the master does **not** stop EKS, which keeps costing money.

Tear everything down, in this order

1. Delete the Argo CD Application `shopkart`.
2. Delete namespace `shopkart`: this removes the load balancer and the Mongo volume.
3. Remove monitoring with helm uninstall monitoring -n monitoring, then delete namespaces monitoring and `argocd`.
4. `terraform destroy` in `infra-eks` (on the master).
5. `terraform destroy` in `infra-master` (from the laptop).
6. Check the AWS console for anything left: instances, volumes, load balancers, NAT gateways.
7. Keep or empty and delete the S3 state bucket.

Rebuilding later: recreate EKS with Terraform (about 20 minutes), create the Mongo Secret, apply the Argo CD Application, and the whole app comes back from Git.

11. Checklist for a future project

The same order works for any similar project: names only, top to bottom.

Prepare

- [ ] AWS account check: identity, region, plan
- [ ] Budget alert
- [ ] Laptop tools: Git, Docker, Node (or your language), AWS CLI, Terraform
- [ ] SSH key for GitHub
- [ ] GitHub repos: app, infra, gitops

Build the app

- [ ] Backend with health, ready and metrics endpoints
- [ ] Frontend
- [ ] Unit tests
- [ ] Dockerfiles
- [ ] docker-compose
- [ ] .gitignore
- [ ] Secret scan (gitleaks)
- [ ] Push to GitHub

CI server

- [ ] SSH key for the server
- [ ] Terraform: IAM role, security group, key pair, EC2
- [ ] Startup script: Java, Jenkins, Docker, Trivy, SonarQube, AWS CLI, Terraform, kubectl, Helm
- [ ] Terraform apply
- [ ] Verify the tools

Jenkins

- [ ] Unlock Jenkins, admin user
- [ ] Plugins: NodeJS, SonarQube Scanner, Stage View
- [ ] Tools: node, sonar-scanner
- [ ] Docker Hub token → credential
- [ ] SonarQube token → credential
- [ ] SonarQube server in Jenkins
- [ ] SonarQube webhook to Jenkins
- [ ] Pipeline job from SCM
- [ ] Jenkinsfile: checkout, test, SonarQube, Trivy fs, build, Trivy image, push
- [ ] First green build

Kubernetes cluster

- [ ] Bootstrap: S3 state bucket
- [ ] Terraform: VPC, EKS, node group, add-ons (CNI, CoreDNS, kube-proxy, Pod Identity, EBS CSI)
- [ ] Terraform apply from the CI server
- [ ] kubeconfig on laptop and server
- [ ] Default StorageClass
- [ ] Test PVC

GitOps

- [ ] Kubernetes YAML in the gitops repo
- [ ] Install Argo CD
- [ ] Database Secret with kubectl
- [ ] Argo CD Application
- [ ] GitHub token → Jenkins credential
- [ ] "Update GitOps repo" stage in the Jenkinsfile
- [ ] Test the full loop

Monitoring

- [ ] Prometheus + Grafana
- [ ] ServiceMonitor for the app
- [ ] Alert rules
- [ ] Dashboards

Finish

- [ ] README in each repo
- [ ] Architecture document
- [ ] Break-fix practice
- [ ] Teardown

12. Teardown and rebuild runbook

All code is in Git, but some things are not: the Jenkins setup, the tokens and the Kubernetes Secrets. After a teardown, follow these phases in this order to bring the whole project back (about 1.5 to 2 hours). The MongoDB data is not kept: the app comes back with an empty database.

What is NOT in Git (must be recreated)

Item	Where it lived	Recreated in
Jenkins plugins, tools, credentials, SonarQube server, job	Master EC2 disk	Phase 2
SonarQube admin password, token, webhook	Master EC2 (SonarQube database)	Phase 2
Docker Hub token, GitHub token	Only inside Jenkins	Phase 2 (create new ones; revoke the old ones)
`terraform.tfvars` (my IP, master IP, laptop role)	Laptop only	Phases 1 and 3
Secret `shopkart-mongo`	Cluster only	Phase 5
Secret `grafana-admin`	Cluster only	Phase 4

Option: keep the master, destroy only EKS

Stopping the master instead of destroying it keeps Jenkins and SonarQube exactly as they are, for about $3 a month of disk. Then a rebuild skips Phases 1 and 2: start the master, run `terraform apply -refresh-only` in `infra-master`, update the **Jenkins URL**

and `admin_cidrs` with the new IP, and continue from Phase 3 (about 45 minutes).

Teardown (in this order)

```
kubectl -n argocd delete application shopkart        # 1. stop Argo CD re-
creating the app
kubectl delete namespace shopkart                    # 2. removes the load
balancer and the Mongo EBS volume
helm uninstall monitoring -n monitoring              # 3. remove Prometheus,
Grafana, Alertmanager
kubectl delete namespace monitoring argocd           # 4. remove the rest
aws elb describe-load-balancers --query
'LoadBalancerDescriptions[].LoadBalancerName'   # 5. must be empty, or
destroy fails
# 6. on the master:  cd ~/shopkart-infra/infra-eks && terraform destroy
(about 15 minutes)
# 7. on the laptop:  cd ~/vs/shopkart-infra/infra-master && terraform destroy
(or stop the instance, see the option above)
# 8. AWS console: check EC2, EBS volumes, load balancers, NAT gateways are
gone
```

Keep the S3 state bucket (it costs almost nothing), so Phase 0 can skip `bootstrap/`.

Phase 0: laptop checks (5 minutes)

```
aws login                                          # fresh AWS session
aws configure get region                           # must be ap-southeast-2
curl -s https://checkip.amazonaws.com              # note my IP; update my_ip
and admin_cidrs if it changed
aws s3 ls | grep shopkart-tfstate                  # state bucket still there?
if not: cd bootstrap && terraform apply
```

Phase 1: master EC2 (15 minutes)

```
cd ~/vs/shopkart-infra/infra-master
cat terraform.tfvars                               # my_ip = "<my-ip>/32",
instance_type = "m7i-flex.large"
terraform init
terraform plan -out tfplan && terraform apply tfplan   # about 1 minute, then
the startup script runs
$(terraform output -raw ssh_command) 'sudo cloud-init status --wait; sudo
tail -3 /var/log/user-data.log'   # wait for "bootstrap completed"
terraform output                                   # note the new Jenkins and
SonarQube URLs
```

Phase 2: Jenkins and SonarQube setup (30 minutes, in the browsers)

1. **Jenkins:** unlock with `sudo cat /var/lib/jenkins/secrets/initialAdminPassword` (on the master), install suggested plugins, create the admin user, set the **Jenkins URL**.
2. **Plugins:** NodeJS, SonarQube Scanner, Pipeline: Stage View.
3. **Tools:** NodeJS `node24`, SonarQube Scanner `sonar-scanner` (both install automatically).
4. **SonarQube** (`:9000`): log in as admin/admin, change the password, create a Global Analysis Token.
5. **Credentials** (System → Global): `sonar-token` (Secret text), `dockerhub` (new Docker Hub Read & Write token), `github-token` (new fine-grained token, `shopkart-gitops` only, Contents read/write).
6. **SonarQube server** in Manage Jenkins → System: name `sonar`, URL `http://localhost:9000`, token `sonar-token`.
7. **Webhook** in SonarQube: `http://172.18.0.1:8080/sonarqube-webhook/`. Check the gateway IP first on the master: `docker network inspect sonarqube_default --format "{{(index .IPAM.Config 0).Gateway}}"`.
8. **Job:** New Item `shopkart-app`, Pipeline script from SCM, `https://github.com/vkashsingh/shopkart-app.git`, branch `*/main`, script `Jenkinsfile`. Click **Build Now** once to register polling.

Phase 3: EKS cluster (25 minutes, mostly waiting)

```
# laptop: put the NEW master IP and my IP in infra-eks/terraform.tfvars
(admin_cidrs), then copy it over
MASTER=$(cd ~/vs/shopkart-infra/infra-master && terraform output -raw
public_ip)
ssh -i ~/.ssh/shopkart-master ubuntu@$MASTER 'git clone
https://github.com/vkashsingh/shopkart-infra.git ~/shopkart-infra'
scp -i ~/.ssh/shopkart-master ~/vs/shopkart-infra/infra-eks/terraform.tfvars
ubuntu@$MASTER:~/shopkart-infra/infra-eks/
ssh -i ~/.ssh/shopkart-master ubuntu@$MASTER

# on the master:
tmux new -s eks                                    # survives SSH drops
cd ~/shopkart-infra/infra-eks
terraform init -backend-config="bucket=shopkart-tfstate-<account-id>"
terraform plan -no-color -out tfplan | grep Plan:   # expect about 60 to add
terraform apply tfplan                             # about 20 minutes
aws eks update-kubeconfig --region ap-southeast-2 --name shopkart-eks
kubectl get nodes                                  # 2 nodes Ready
kubectl apply -f k8s/storageclass-gp3.yaml         # default StorageClass

# laptop:
aws eks update-kubeconfig --region ap-southeast-2 --name shopkart-eks --alias
shopkart-eks   # laptop kubectl access
```

Phase 4: monitoring BEFORE the app (10 minutes)

Install monitoring first: the GitOps repo contains a ServiceMonitor and a PrometheusRule, and Kubernetes only knows those kinds after the Helm chart installs them. Otherwise the first Argo CD sync fails on those two files.

```
cd ~/vs/shopkart-gitops && git pull --rebase
kubectl create namespace monitoring
kubectl -n monitoring create secret generic grafana-admin \
  --from-literal=admin-user=admin --from-literal=admin-password="$(openssl
rand -hex 16)"   # Grafana login
helm repo add prometheus-community https://prometheus-
community.github.io/helm-charts && helm repo update
helm upgrade --install monitoring prometheus-community/kube-prometheus-stack
\
  -n monitoring --version 92.16.1 -f platform/monitoring/values.yaml --wait -
-timeout 10m   # don't interrupt
kubectl -n monitoring get pods                     # 7 pods Running
```

Phase 5: Argo CD and the app (15 minutes)

```
kubectl create namespace argocd
kubectl apply -n argocd --server-side -f
https://raw.githubusercontent.com/argoproj/argo-
cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-server --timeout=300s
kubectl create namespace shopkart
MONGO_USER=shopkart; MONGO_PASS=$(openssl rand -hex 16)
kubectl -n shopkart create secret generic shopkart-mongo \
  --from-literal=mongo-root-username="$MONGO_USER" --from-literal=mongo-root-
password="$MONGO_PASS" \
  --from-literal=mongo-
url="mongodb://$MONGO_USER:$MONGO_PASS@mongo:27017/shopkart?authSource=admin"
unset MONGO_PASS
kubectl apply -n argocd -f apps/shopkart-app.yaml   # Argo CD deploys the
app, ServiceMonitor and alerts from Git
kubectl -n argocd get application shopkart        # wait for Synced / Healthy
echo "http://$(kubectl -n shopkart get svc shop-frontend -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}')"   # open after 2-3
minutes
```

The app starts with the last image tags in `shopkart-gitops`, which still exist on Docker Hub, so no Jenkins build is needed to bring it up.

Phase 6: prove it works (10 minutes)

1. Open the site, add a product.
2. Prometheus (`port-forward ... 9090:9090`): target `shop-backend` 2/2 UP, group `shopkart.rules` loaded.
3. Push a small change to `shopkart-app`: Jenkins builds, commits the tag to `shopkart-gitops`, Argo CD rolls it out.

13. Command cheat sheet

Every command used in this project, grouped by tool, each with a one-line comment. Words in angle brackets are placeholders. Unless a block says "on the master", run it in a terminal on the laptop.

AWS account and laptop

```
aws login                                          # sign in the AWS CLI
(short-lived session; repeat on ExpiredToken)
aws sts get-caller-identity                        # which identity am I using?
(must not be root)
aws configure set region ap-southeast-2            # set the default region
allowed for this account
aws ec2 describe-instance-types --filters Name=free-tier-eligible,Values=true
\
  --query 'InstanceTypes[].InstanceType' --output text   # instance types the
Free plan allows
aws eks describe-cluster-versions --query 'clusterVersions[?
versionStatus==`STANDARD_SUPPORT`].clusterVersion' --output text  # supported
EKS versions
curl -s https://checkip.amazonaws.com              # my public IP (for security
groups and admin_cidrs)
```

SSH keys and GitHub

```
ssh-keygen -t ed25519 -C "<label>" -f ~/.ssh/<name>       # create a key pair
(private + .pub)
ssh-add --apple-use-keychain ~/.ssh/<name>              # load the key into
the macOS Keychain (no passphrase prompts)
pbcopy < ~/.ssh/id_ed25519.pub                          # copy the public key
to paste into GitHub
ssh -T git@github.com                                   # test GitHub SSH
access
```

Git

```
git init -b main                                    # start a new repo
git add . && git commit -m "<message>"              # stage and commit
git remote add origin git@github.com:vkashsingh/<repo>.git   # link to GitHub
git push -u origin main                             # first push
git pull --rebase                                   # get remote commits first
(Jenkins writes to shopkart-gitops)
git check-ignore -v <file>                          # prove a file is ignored
(tfvars, state)
git reset HEAD~1                                    # undo the last commit that
was NOT pushed
git revert <sha>                                    # undo a pushed commit
(GitOps rollback)
docker run --rm -v "$PWD:/repo" ghcr.io/gitleaks/gitleaks:latest git /repo --
redact   # secret scan before pushing
```

Docker and the app on the laptop

```
docker compose up -d --build                        # build and start frontend,
backend, Mongo
docker compose ps                                   # status and health of each
service
docker compose exec shop-backend whoami             # check the container is
not root
docker compose down                                 # stop and remove
containers (named volume keeps the data)
docker manifest inspect vkashs/shopkart-backend:<tag>   # does this tag exist
on Docker Hub?
lsof -nP -iTCP:<port> -sTCP:LISTEN                # which process is holding a
port?
```

Terraform

```
terraform init                                      # download providers and
modules
terraform init -backend-config="bucket=<state-bucket>"   # init with the S3
backend (infra-eks)
terraform validate                                  # check syntax and
references
terraform plan -no-color -out tfplan | grep -E 'Plan:|will be destroyed'   #
save a plan and show the summary
terraform apply tfplan                              # apply exactly the saved
plan
terraform apply -refresh-only -auto-approve       # update outputs (e.g. new
public IP) without changing resources
terraform apply -replace=aws_instance.master      # rebuild the EC2 (needed
after a user-data change)
terraform apply -target='<address>'               # create one resource only
(recovery use only)
terraform output -raw ssh_command                 # print one output value
terraform state list                              # what Terraform is
tracking
terraform providers lock -platform=darwin_arm64 -platform=linux_amd64   #
lock file for Mac and Linux
terraform destroy                                 # delete everything in this
folder
```

The master EC2 (Jenkins and SonarQube server)

```
$(terraform output -raw ssh_command)                                      #
SSH to the master (from infra-master)
$(terraform output -raw ssh_command) 'sudo cloud-init status --wait'     #
wait for the startup script to finish
$(terraform output -raw ssh_command) 'sudo tail -30 /var/log/user-data.log'
# read the startup script log
sudo cat /var/lib/jenkins/secrets/initialAdminPassword                    #
Jenkins unlock password (on the master)
sudo -u jenkins docker ps                                                 # can
the jenkins user use Docker? (on the master)
aws ec2 stop-instances  --instance-ids <master-id>                        #
pause the master (disk keeps everything)
aws ec2 start-instances --instance-ids <master-id>                        #
start it again (new public IP)
tmux new -s eks      # start a session that survives SSH drops (on the
master)
tmux attach -t eks   # reconnect to it
```

Kubernetes and EKS

```
aws eks update-kubeconfig --region ap-southeast-2 --name shopkart-eks   # add
the cluster to kubeconfig
kubectl config get-contexts                                             # list
clusters (* = current)
kubectl config use-context shopkart-eks                                 #
switch cluster (or minikube)
kubectl get nodes -o wide                                               # node
status, version, IPs
kubectl get pods -A                                                     #
every pod in every namespace
kubectl -n shopkart get pods,pvc,svc                                    # the
app's pods, volumes, services
kubectl -n shopkart describe pod <pod>                                  #
events: why Pending, OOMKilled, probe failures
kubectl -n shopkart logs <pod> --previous                               # logs
of the container before its last restart
kubectl -n shopkart get deploy -o 'custom-
columns=NAME:.metadata.name,IMAGE:.spec.template.spec.containers[0].image'
# running image tags
kubectl describe nodes | grep -A6 "Allocated resources"                 #
requested CPU and memory per node
kubectl create secret generic shopkart-mongo -n shopkart --from-literal=
<key>=<value>   # create a Secret outside Git
kubectl -n shopkart describe secret shopkart-mongo                      # see
key names and sizes, not values
kubectl apply -f k8s/storageclass-gp3.yaml                              #
default gp3 StorageClass
kubectl apply --dry-run=client -f manifests/                            #
validate YAML without changing anything
```

Argo CD

```
kubectl apply -n argocd --server-side -f <argo-cd install.yaml>             #
install Argo CD
kubectl -n argocd port-forward svc/argocd-server 8443:443                   #
open the UI at https://localhost:8443
kubectl -n argocd get secret argocd-initial-admin-secret -o
jsonpath='{.data.password}' | base64 -d; echo   # first admin password
kubectl apply -n argocd -f apps/shopkart-app.yaml                           #
register the app (one time)
kubectl -n argocd get application shopkart                                  #
Synced / Healthy?
kubectl -n argocd annotate application shopkart
argocd.argoproj.io/refresh=normal --overwrite   # check Git now instead of in
3 minutes
```

Helm and monitoring

```
helm repo add prometheus-community https://prometheus-
community.github.io/helm-charts   # add the chart repo
helm repo update                                                 # refresh
the chart list
helm upgrade --install monitoring prometheus-community/kube-prometheus-stack
\
  -n monitoring --version 92.16.1 -f platform/monitoring/values.yaml --wait -
-timeout 10m   # install or upgrade (don't interrupt)
helm -n monitoring history monitoring                             # every
revision and its status
helm -n monitoring rollback monitoring <revision>                # go back
to a working revision
helm uninstall monitoring -n monitoring                           # remove
the stack (teardown)
kubectl -n monitoring port-forward svc/monitoring-kube-prometheus-prometheus
9090:9090      # Prometheus at localhost:9090
kubectl -n monitoring port-forward svc/monitoring-grafana 3001:80
# Grafana at localhost:3001
kubectl -n monitoring port-forward svc/monitoring-kube-prometheus-
alertmanager 9093:9093   # Alertmanager at localhost:9093
kubectl -n monitoring get secret grafana-admin -o jsonpath='{.data.admin-
password}' | base64 -d; echo   # Grafana password
kubectl -n shopkart get servicemonitor,prometheusrule            #
monitoring objects applied by Argo CD
curl -s localhost:9090/api/v1/rules | grep -o '"name":"shopkart.rules"'   #
did Prometheus load our rules?
```

Testing and checks

```
LB=$(kubectl -n shopkart get svc shop-frontend -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}')   # load balancer
address
curl -s http://$LB/api/products                                    # is the
app answering?
for i in $(seq 1 50); do curl -s -o /dev/null http://$LB/api/products; done
# generate traffic for metrics
kubectl -n shopkart scale deploy shop-backend --replicas=5         # self-
heal test (Argo CD puts it back to 2)
kubectl -n shopkart delete pod mongo-0                             #
persistence test (data must survive)
kubectl -n shopkart exec <backend-pod> -- kill 1                   # restart
a container (alert test)
```

zsh habits

```
kubectl get deploy -o 'custom-columns=...[0]...'   # quote anything with [ ]
* ? or zsh treats it as a file pattern
echo 'setopt interactivecomments' >> ~/.zshrc      # allow pasted # comments
at the prompt
# avoid ! in commands at the prompt: zsh treats it as history expansion
```